

Sin el modelo de datos no hay aplicación

Informe de viabilidad · Acceso al repositorio original y a Supabase · Intranet Black Moose Group · 9 de septiembre de 2026

CASTELLANO

Esta intranet no tiene backend propio: el backend es Supabase. Eso significa que las reglas del negocio —qué entidades existen, cómo se relacionan, quién puede ver qué, cómo se calcula una comisión o cuándo se descuenta el stock— no viven en un servidor que podamos deducir mirando la pantalla. Viven *dentro* de la base de datos: en sus tablas, sus relaciones, sus políticas de acceso y sus funciones.

Nosotros hemos copiado lo único que se ve: la fachada. Y la fachada es la capa que menos valor tiene, porque es la que menos cuesta rehacer.

La consecuencia es dura y hay que decirla clara: nuestra copia no funciona, y no puede llegar a funcionar por mucho que sigamos copiando. No es cuestión de terminar el 29 % que falta. El 71 % que ya tenemos tampoco ejecuta una sola operación real. Sin acceso al banco de entidades estamos copiando por copiar: pintando un cuadro muy fiel de una máquina que no podemos encender.

1. Qué es realmente esta intranet

El navegador del live guarda un token de sesión de Supabase. Eso no es un detalle técnico menor: es la prueba de que **el cliente habla directamente con la base de datos**, sin una capa de API propia por medio. En una arquitectura así —Supabase: PostgreSQL, PostgREST, Auth, Storage y políticas de seguridad a nivel de fila— **el esquema de la base de datos es la aplicación**. No es documentación de la aplicación: es la aplicación.

Y si además existieran funciones de servidor, tampoco las veríamos. Eso no debilita el argumento: lo refuerza. Todo lo que decide el comportamiento está en el lado que no podemos mirar.

Dicho de otra manera: de una captura de pantalla se puede deducir **cómo se ve** un pedido. No se puede deducir qué es un pedido, con qué se relaciona, qué estados admite, qué pasa al confirmarlo, ni quién tiene permiso para hacerlo. **Eso último es el producto**. Lo primero es su fotografía.

2. Qué hemos copiado y qué no

CAPA	¿LA TENEMOS?	DETALLE
Presentación (maquetación, color, tipografía)	Sí · 71 %	63 de las 89 pantallas del live, calcadas con evidencia medida.
Navegación y estructura	Sí	12 módulos, rutas y sub-navegaciones.
Textos y microcopy	Sí	Literales tomados del live.

Modelo de entidades (tablas y relaciones)	No · 0 %	No conocemos ni una tabla. Lo que intuimos está deducido de cómo se ven las columnas por fuera.
Lógica de negocio	No · 0 %	Cálculos, estados, validaciones, disparadores.
Permisos y seguridad (RLS)	No · 0 %	Quién ve qué. Hemos calcado las 11 cajas de permisos <i>de la pantalla</i> ; no sabemos qué hacen.
Autenticación real	No	Nuestro login es decorativo.
Integraciones	No · 0 de 5	Holded, Songstats, DocuSeal, Google Calendar y Supabase Storage.

Las cuatro primeras filas son la fachada. Las cinco últimas son la aplicación.

3. El impacto real: lo que ya ha pasado

No es un riesgo futuro. Es el estado actual, después de dos meses de trabajo intensivo y de un resultado que, en su propia categoría, es bueno:

37.315 LÍNEAS DE CÓDIGO	1.019 TESTS EN VERDE	472 COMMITTS · 2 MESES	0 OPERACIONES REALES
-----------------------------------	--------------------------------	----------------------------------	--------------------------------

MEDIDA	HOY	QUÉ SIGNIFICA
Acciones de alta catalogadas en el live	30	<i>Nuevo pedido, Nuevo tour, Nueva creatividad, Nuevo contrato, Nueva campaña...</i> Ninguna es ejecutable en nuestra copia. Todos los botones son inertes.
Métodos del adaptador de datos implementados	0 de 17	Los 17 lanzan <code>not implemented</code> , porque no sabemos contra qué tablas hablarían.
Módulos conectados a un repositorio de datos	5 de 16	Los otros 11 leen literales escritos dentro del propio componente.
Líneas de datos escritas a mano	7.966	En 97 ficheros. Ninguna procede de una base de datos: todas se han tecleado desde una captura.
Capturas archivadas como única especificación	413	99 MB. Si el live cambia, envejecen sin avisar.
Barridos completos del live	5	Una jornada de trabajo cada uno, y hay que repetirlos porque el live se mueve.

El coste de oportunidad ya consumido. Dos meses de ingeniería han producido una maqueta de altísima fidelidad. Ese trabajo tiene valor real como validación de diseño y como sistema de componentes. Pero **no ha producido ni un gramo de producto funcional**, y no lo hará mientras el modelo de datos siga siendo invisible. Cada semana que sigamos así, ampliamos la fachada y no movemos el producto.

4. Las reglas que una captura nunca enseña

Esto no es abstracto. El propio live nos dice, en sus textos, que detrás de cada pantalla hay una regla que nosotros no podemos ver ni reproducir:

MÓDULO	LO QUE DICE EL LIVE	LO QUE NO SABEMOS
CRUDA · Costes	«Escandallo vivo: el PVP de cada prenda se recalcula al cambiar un tejido, el tinte o los parámetros.»	La fórmula. Cuatro precios por modelo que salen de un cálculo que no tenemos.
CRUDA · Pedidos	«El stock se descuenta al confirmar el pedido, no al crearlo.»	El disparador, la reserva y qué pasa con dos comerciales vendiendo la misma unidad.
ConceptOne · Cobros	«Total y vencimientos salen del plan de pagos; lo cobrado, de las facturas conciliadas en Holded.»	El plan de pagos como entidad, y la conciliación contra un ERP externo.
ConceptOne · Liquidaciones	«Estado de dinero de cada show: cobros del promotor, gastos, y lo liquidado al artista.»	Cómo se encadenan show → cobro → gasto → liquidación. Son cuatro entidades y tres relaciones.
Euphoric · Rendimiento	«Qué deja cada cuenta, con las horas reales de los partes. Objetivo: margen 55 % · 65 €/h.»	El cálculo del margen y el semáforo que clasifica cada cuenta.
Euphoric · Facturación	«Emite la proforma del mes desde sus suscripciones activas y, al cerrar, conviértela en factura.»	Suscripciones, proformas y el puente con Holded.
Management · Insights	«Todo se calcula del histórico — nada se introduce a mano.»	El histórico, y una API de pago por uso (Songstats) que no tenemos.
Team · Usuarios	«Los permisos se editan en su ficha, no en una lista aparte.»	El modelo de permisos entero. Hemos copiado las cajas; no sabemos qué autorizan.

Podemos dibujar las ocho pantallas con fidelidad de píxel. **No podemos ejecutar ninguna de las ocho reglas.**

5. Si el live se cae ahora

Lo que se suele suponer: «tenemos una copia local, algo funcionará».

Lo que pasaría de verdad: la aplicación arrancararía y no serviría para nada. **Ninguna sección sería utilizable**, porque ninguna sección tiene detrás datos reales ni las reglas que los gobiernan. Y al mismo tiempo perderíamos la única especificación que existe del producto, que es la propia pantalla del live.

SECCIÓN	QUÉ DEJARÍA DE SER POSIBLE
ConceptOne · Bookings	Crear o cerrar un show, mover su estado en el pipeline, generar un contrato, cobrar, conciliar un gasto o liquidar a un artista.
ConceptOne · Management	Todo: el roster depende de una API externa y de un histórico que no tenemos.
Euphoric Media	Imputar horas, calcular márgenes, emitir una proforma o facturar.
CRUDA	Calcular un precio, confirmar un pedido o descontar stock.
CRM	Mover una oportunidad de etapa o medir un pipeline sin oportunidades reales.

Mixmag · TAGMAG	Planificar, vender o publicar: no hay tarifario ni calendario reales.
Team · Configuración	Dar de alta a una persona o concederle un permiso: no hay usuarios ni sistema de acceso.
Creativos · Mi trabajo	Asignar, aprobar o entregar una creatividad: no hay ficheros, ni avatares, ni flujo de aprobación.

El punto que importa: el problema no es el 29 % de pantallas que faltan. Es que el 71 % que ya tenemos *tampoco funciona*. Y reconstruir el modelo de datos y las reglas de 89 pantallas a base de mirar capturas antiguas no es «terminar el proyecto»: es inventarse otro producto parecido, con meses de trabajo por delante y sin ninguna garantía de que se comporte como el original.

6. Qué pedimos

En orden de utilidad frente a coste de concesión. El primero es, con diferencia, el que más desbloquea:

#	PETICIÓN	ALCANCE MÍNIMO SUFICIENTE	QUÉ DESBLOQUEA
1	Esquema de la base de datos	Volcado de estructura sin datos (<code>pg_dump --schema-only</code>): tablas, relaciones, restricciones, vistas, funciones y políticas RLS.	Convierte 17 excepciones en una aplicación. No expone ni un dato personal.
2	Lectura del repositorio original	Acceso de solo lectura, o un volcado del código en un momento dado.	La lógica de negocio y el fin de los barridos: un <code>git diff</code> sustituye a una jornada de capturas.
3	Supabase en lectura	Usuario de solo lectura contra <i>staging</i> o una réplica, con RLS activo. No hace falta producción.	Datos reales y verificación de extremo a extremo.

Garantías por nuestra parte: acceso de solo lectura, nunca escritura; claves fuera del repositorio; datos personales fuera de los *seeds* y de las capturas; y el modo local se mantiene por defecto para desarrollo y tests, que seguirán corriendo sin tocar ningún sistema real.

7. Recomendación

Pedir el esquema de la base de datos, y pedirlo ahora. Es lo más barato de conceder —no contiene información de nadie— y es lo único que convierte dos meses de fachada en una aplicación que puede funcionar. Sin él, cualquier trabajo adicional sigue siendo decoración: más pantallas bonitas sobre una máquina que no arranca.

Y conviene pedirlo mientras el live sigue accesible, para poder contrastar lo que recibamos con lo que ya tenemos medido. Hacerlo después de una caída sería empezar la casa por el tejado.

Elaborado el 9 de septiembre de 2026. Fuentes: barrido del live del mismo día (89 rutas inventariadas, 23 pantallas de ConceptOne capturadas), informe de deltas del 30 de julio de 2026, y métricas del repositorio `Boilerplate` en `main` (`990dd7e`). Las cifras de los apartados 2 y 3 son medidas sobre el repositorio y sobre las capturas, no estimaciones. Las citas del apartado 4 son textos literales del live.

Without the data model there is no application

Feasibility report · Access to the original repository and to Supabase · Black Moose Group intranet · 9 September 2026

ENGLISH

This intranet has no backend of its own: Supabase is the backend. That means the business rules — which entities exist, how they relate, who may see what, how a commission is calculated, when stock is deducted — do not live in a server we could infer by looking at the screen. They live *inside* the database: in its tables, its relations, its access policies and its functions.

We have copied the only thing that is visible: the façade. And the façade is the layer with the least value, because it is the cheapest one to rebuild.

The consequence is blunt and worth stating plainly: our copy does not work, and it cannot be made to work by copying more. This is not about finishing the missing 29 %. The 71 % we already have does not execute a single real operation either. Without access to the entity model we are copying for the sake of copying: painting a very faithful picture of a machine we cannot switch on.

1. What this intranet actually is

The live app stores a Supabase session token in the browser. That is not a minor technical detail: it is proof that **the client talks straight to the database**, with no API layer of its own in between. In an architecture like that — Supabase: PostgreSQL, PostgREST, Auth, Storage and row-level security — **the database schema *is* the application**. It is not documentation about the application: it is the application.

And if server-side functions did exist, we would not see those either. That does not weaken the argument; it strengthens it. Everything that decides behaviour sits on the side we cannot look at.

Put differently: from a screenshot you can infer **what an order looks like**. You cannot infer what an order *is*, what it relates to, which states it allows, what happens when it is confirmed, or who is allowed to do it. **That is the product**. The former is a photograph of it.

2. What we have copied, and what we have not

LAYER	DO WE HAVE IT?	DETAIL
Presentation (layout, colour, type)	Yes · 71 %	63 of the live's 89 screens, replicated against measured evidence.
Navigation and structure	Yes	12 modules, routes and sub-navigation.
Copy and microcopy	Yes	Taken verbatim from the live.

Entity model (tables and relations)	No · 0 %	We do not know a single table. What we guess is inferred from how columns look from the outside.
Business logic	No · 0 %	Calculations, states, validations, triggers.
Permissions and security (RLS)	No · 0 %	Who sees what. We replicated the 11 permission boxes <i>on screen</i> ; we do not know what they do.
Real authentication	No	Our login is decorative.
Integrations	No · 0 of 5	Holded, Songstats, DocuSeal, Google Calendar and Supabase Storage.

The first four rows are the façade. The last five are the application.

3. The real impact: what has already happened

This is not a future risk. It is the current state, after two months of intense work and a result that, within its own category, is good:

37,315 LINES OF CODE	1,019 PASSING TESTS	472 COMMITTS · 2 MONTHS	0 REAL OPERATIONS
--------------------------------	-------------------------------	-----------------------------------	-----------------------------

MEASURE	TODAY	WHAT IT MEANS
Create actions catalogued in the live	30	<i>New order, New tour, New artwork, New contract, New campaign...</i> None is executable in our copy. Every button is inert.
Data-adapter methods implemented	0 of 17	All 17 throw <code>not implemented</code> , because we do not know which tables they would talk to.
Modules wired to a data repository	5 of 16	The other 11 read literals written inside the component itself.
Lines of hand-typed data	7,966	Across 97 files. None comes from a database: every one was typed from a screenshot.
Screenshots kept as our only specification	413	99 MB. When the live changes, they go stale without warning.
Full sweeps of the live	5	A working day each, and they must be repeated because the live keeps moving.

The opportunity cost already spent. Two months of engineering have produced a very high-fidelity mock-up. That work has real value as design validation and as a component system. But **it has produced no working product whatsoever**, and it will not while the data model stays invisible. Every week we continue like this, we widen the façade and move the product not at all.

4. The rules a screenshot never shows

This is not abstract. The live itself tells us, in its own copy, that behind every screen there is a rule we can neither see nor reproduce:

MODULE	WHAT THE LIVE SAYS	WHAT WE DO NOT KNOW
--------	--------------------	---------------------

CRUDA · Costing	"Live costing: the retail price of each garment is recalculated when a fabric, a dye or a parameter changes."	The formula. Four prices per model produced by a calculation we do not have.
CRUDA · Orders	"Stock is deducted when the order is confirmed, not when it is created."	The trigger, the reservation, and what happens when two salespeople sell the same last unit.
ConceptOne · Receivables	"Total and due dates come from the payment plan; what has been collected, from invoices reconciled in Helded."	The payment plan as an entity, and reconciliation against an external ERP.
ConceptOne · Settlements	"The money status of each show: promoter payments, expenses, and what has been settled with the artist."	How show → payment → expense → settlement chain together. Four entities and three relations.
Euphoric · Performance	"What each account leaves, using the real hours from timesheets. Target: 55 % margin · €65/h."	The margin calculation and the traffic-light that classifies each account.
Euphoric · Invoicing	"Issue the month's proforma from active subscriptions and, on closing, turn it into an invoice."	Subscriptions, proformas, and the bridge to Helded.
Management · Insights	"Everything is computed from history — nothing is entered by hand."	The history, and a pay-per-use API (Songstats) we do not have.
Team · Users	"Permissions are edited on the person's record, not in a separate list."	The whole permission model. We copied the boxes; we do not know what they authorise.

We can draw all eight screens to the pixel. **We cannot execute a single one of the eight rules.**

5. If the live goes down now

The usual assumption: "we have a local copy, something will work."

What would actually happen: the application would start and be useless. **No section would be usable**, because no section has real data behind it, nor the rules that govern that data. And at the same time we would lose the only specification of the product that exists, which is the live screen itself.

SECTION	WHAT WOULD STOP BEING POSSIBLE
ConceptOne · Bookings	Creating or closing a show, moving it through the pipeline, generating a contract, collecting payment, reconciling an expense or settling with an artist.
ConceptOne · Management	Everything: the roster depends on an external API and on history we do not have.
Euphoric Media	Logging hours, computing margins, issuing a proforma or invoicing.
CRUDA	Calculating a price, confirming an order or deducting stock.
CRM	Moving an opportunity through stages, or measuring a pipeline with no real opportunities.

Mixmag · TAGMAG	Planning, selling or publishing: there is no real rate card and no real calendar.
Team · Settings	Adding a person or granting a permission: there are no users and no access system.
Creative · My work	Assigning, approving or delivering artwork: no files, no avatars, no approval flow.

The point that matters: the problem is not the 29 % of screens still missing. It is that the 71 % we already have *does not work either*. And rebuilding the data model and the rules of 89 screens from old screenshots is not "finishing the project": it is inventing a different, similar product, with months of work ahead and no guarantee it behaves like the original.

6. What we are asking for

In order of value against cost of granting. The first is by far the one that unblocks the most:

#	REQUEST	MINIMUM SUFFICIENT SCOPE	WHAT IT UNBLOCKS
1	The database schema	A structure-only dump, no data (<code>pg_dump --schema-only</code>): tables, relations, constraints, views, functions and RLS policies.	Turns 17 exceptions into an application. It exposes no personal data at all.
2	Read access to the original repository	Read-only access, or a point-in-time dump of the code.	The business logic, and the end of the sweeps: a <code>git diff</code> replaces a full day of screenshotting.
3	Read access to Supabase	A read-only user against staging or a replica, with RLS enabled. Production is not required.	Real data and end-to-end verification.

Our guarantees: read-only access, never write; credentials kept out of the repository; personal data kept out of seeds and screenshots; and local mode stays the default for development and tests, which will keep running without touching any real system.

7. Recommendation

Ask for the database schema, and ask now. It is the cheapest thing to grant — it contains nobody's information — and it is the only thing that turns two months of façade into an application that can actually run. Without it, any further work remains decoration: more handsome screens on top of a machine that will not start.

And it is worth asking while the live is still reachable, so we can check whatever we receive against what we have already measured. Doing it after an outage would be building the roof before the walls.

Prepared on 9 September 2026. Sources: the live sweep of the same day (89 routes inventoried, 23 ConceptOne screens captured), the delta report of 30 July 2026, and metrics from the `Boilerplate` repository on `main` (`990dd7e`). The figures in sections 2 and 3 are measurements of the repository and of the captures, not estimates. The quotations in section 4 are verbatim text from the live, translated.